

SOVEREIGN AGENT KEYS

AI Agent Utility Upgrade — Full Implementation Prompt

OG APAC Hackathon 2026 · Track 5: Privacy & Sovereign Infrastructure · v4.0

WHAT THIS PROMPT DOES

This document is a complete implementation brief for your AI coding agent. It upgrades the SAK project from a pure infrastructure demo into a **fully functional AI agent platform** where users connect their wallet, spawn sovereign agents they own on-chain, and issue real instructions that flow through ZK proof enforcement, OG DA memory, and on-chain settlement.

The prompt covers 5 modules in strict dependency order. Give your agent one module at a time. Verify each compiles before moving to the next.

SCOPE OVERVIEW

Module	What Changes	Impact
1. Smart Contract	Add executeTask() + wallet ownership + task history	Agents do real on-chain work
2. Orchestrator	Add task-parser.ts + --execute-task CLI flag	Natural language → structured intent
3. API Route	New /api/execute-task + /api/get-tasks routes	Frontend can trigger agent actions
4. Wallet Connect	RainbowKit + wagmi — connect button in header	Users own their agents
5. Task UI	TaskPanel component in AgentDetailModal	Users give agents instructions

MODULE 1 OF 5

Smart Contract — AgentRegistry.sol

TYPE: Development

FILE:
contracts/contracts/AgentRegistry.sol

PRIORITY: Do first

CONTEXT

Currently AgentRegistry only has logIntent() which records that an agent WOULD DO something — but never does anything. We need to replace this with executeTask() which: (1) verifies a ZK proof on-chain, (2) records the full task with type + result + DA hash, and (3) increments the agent's task counter. We also need wallet ownership so each agent belongs to a specific user address.

EXACT CHANGES TO MAKE

1 Update the Agent struct

Add these fields to the Agent struct:

```
uint256 taskCount; // increments on every successful executeTask()
uint256 spawnedAt; // block.timestamp at registration
Keep all existing fields (owner, pubKeyHash, constitutionHash, isActive).
```

2 Add a Task struct

Add this new struct above the state variables:

```
struct Task {
    uint256 agentId;
    address submittedBy;
    string taskType; // 'transfer' | 'swap' | 'message' | 'custom'
    string intentDataId; // 0G DA root hash
    string result; // short outcome written by orchestrator
    uint256 executedAt;
    bool success;
}
```

3 Add state mappings

Add these three new mappings:

```
uint256 public nextTaskId;
mapping(uint256 => Task) public tasks;
mapping(uint256 => uint256[]) public agentTaskHistory; // agentId => taskIds
mapping(address => uint256[]) public ownerAgents; // owner => agentIds
Set nextTaskId = 1 in the constructor.
```

4 Update registerAgent() signature

Add a third parameter: address agentOwner

Inside the function, set agents[agentId].spawnedAt = block.timestamp and push agentId into ownerAgents[agentOwner].

The orchestrator backend will pass the user's connected wallet address here.

5 Replace logIntent() with executeTask()

DELETE the entire logIntent() function.

ADD executeTask() with this signature:

```
function executeTask(
  uint256 agentId,
  string memory taskType,
  string memory intentDataId,
  string memory result,
  uint[2] memory pA,
  uint[2][2] memory pB,
  uint[2] memory pC,
  uint[4] memory pubSignals
) external onlyAgentOwner(agentId) agentMustBeActive(agentId) returns (uint256)
```

Inside the function:

```
a) bool isValid = verifier.verifyProof(pA, pB, pC, pubSignals);
require(isValid, 'ZK Proof is invalid');
b) Create and store the Task struct with nextTaskId++
c) agents[agentId].taskCount++
d) agentTaskHistory[agentId].push(taskId)
e) emit TaskExecuted(...)
f) return taskId
```

6 Add TaskExecuted event

```
event TaskExecuted(
  uint256 indexed taskId,
  uint256 indexed agentId,
  address indexed owner,
  string taskType,
  string intentDataId,
  bool success
);
```

7 Add view helpers

Add these three view functions:

```
function getTask(uint256 taskId) external view returns (Task memory)
function getAgentTaskHistory(uint256 id) external view returns (uint256[] memory)
function getOwnerAgents(address owner_) external view returns (uint256[] memory)
```

8 Redeploy

cd contracts

npx hardhat run scripts/deploy.ts --network 0g-testnet

Update addresses.json and README.md with new contract addresses.

Run: npx hardhat compile first to check for errors.

TYPE: Development	FILES: ai-orchestrator/src/task-parser.ts + agent.ts	Run: npm install after changes
-------------------	--	--------------------------------

CONTEXT

The orchestrator currently only does genesis (spawn). It never receives an instruction from a user and does nothing useful after spawning. We need to add two things: (1) a task-parser.ts that converts natural language instructions into structured intents the ZK circuit can prove, and (2) update agent.ts to accept a --execute-task flag and run a full task execution pipeline.

TASK A — Create ai-orchestrator/src/task-parser.ts

FILE: ai-orchestrator/src/task-parser.ts — CREATE NEW

This file exports one async function:

```
parseTaskInstruction(instruction: string, taskType: string, maxAmount: number) returns  
Promise<{ amount: number, targetAddress: string, description: string }>
```

Logic inside the function:

1. Use a simple rule-based parser (no AI API needed for the hackathon): a) Extract amount: look for patterns like '50 tokens', 'send 100', '0.5 ETH' Use regex: `/(?:send|transfer|swap|pay)?\s*(\d+(?:\.\d+)?)\s*(?:tokens?|eth|0g|usdc)?/i` If no amount found, default to `Math.floor(maxAmount * 0.1)` Cap at `maxAmount` (constitutional limit) b) Extract address: look for `0x...` patterns Use regex: `/0x[a-fA-F0-9]{40}/` If none found, use a default testnet address: `'0x70997970C51812dc3A010C7d01b50e0d17dc79C8'` c) Build description string: e.g. `'Agent executed: transfer 50 tokens to 0x7099...'` 2. Return `{ amount, targetAddress, description }`

NOTE: For a stronger demo, optionally add a Claude API call here: POST to <https://api.anthropic.com/v1/messages> with the instruction and ask it to return JSON: `{ amount, targetAddress, description }`. Use `ANTHROPIC_API_KEY` from `.env`. Fall back to rule-based if API fails.

TASK B — Update ai-orchestrator/src/agent.ts

Add these imports at the top:

```
import { parseTaskInstruction } from '../task-parser'; import * as fs from 'fs'; import *  
as path from 'path';
```

Add TaskPayload interface:

```
interface TaskPayload { agentId: number; taskType: string; instruction: string;  
ownerAddress: string; maxAmount: number; timestamp: number; }
```

Add executeTask() method to the SovereignAgent class:

```

async executeTask(payload: TaskPayload): Promise<void> { // 1. Parse instruction into
structured intent const intent = await parseTaskInstruction( payload.instruction,
payload.taskType, payload.maxAmount ); // 2. Log full intent payload to 0G DA const
intentPayload = JSON.stringify({ ...payload, intent, ts: Date.now() }); const daRootHash =
await this.zgService.logIntentMemory( intentPayload,
`task-${payload.agentId}-${Date.now()}` ); // 3. Generate real Groth16 ZK proof const {
pA, pB, pC, pubSignals } = await generateProof( intent.amount, intent.targetAddress,
payload.maxAmount, intent.targetAddress ); // 4. Call AgentRegistry.executeTask() on 0G
Chain const tx = await this.registry.executeTask( payload.agentId, payload.taskType,
daRootHash, intent.description.slice(0, 120), pA, pB, pC, pubSignals ); const receipt =
await this.waitForReceipt(tx.hash); // 5. Print structured result for API to parse
console.log(`TASK_RESULT:${JSON.stringify({ taskId, agentId: payload.agentId, txHash:
tx.hash, daRootHash, result: intent.description })}`); }

```

Update the CLI entry point (main function) to handle --execute-task flag:

```

if (args.includes('--execute-task')) { const payloadPath = process.env.TASK_PAYLOAD_PATH
|| path.resolve(__dirname, '..', 'task_input.json'); const payload =
JSON.parse(fs.readFileSync(payloadPath, 'utf-8')); await agent.executeTask(payload);
return; }

```

Update registerAgent() call inside spawn() to pass owner address:

```

// OLD: const tx = await this.registry.registerAgent(pubKeyHash, constitutionHash); // NEW
- pass the user's wallet address as third argument: const ownerAddress =
process.env.SPAWN_OWNER_ADDRESS || this.wallet.address; const tx = await
this.registry.registerAgent( pubKeyHash, constitutionHashBytes32, ownerAddress );

```

Update the REGISTRY_ABI constant to include the new executeTask() and getOwnerAgents() function signatures matching the updated contract.

MODULE 3 OF 5

API Routes — execute-task + get-tasks + update spawn

TYPE: Development

FILES: mission-control/src/app/api/

3 route files to create/update

CREATE api/execute-task/route.ts

POST handler. Reads body: { agentId, taskType, instruction, ownerAddress, maxAmount }. Validates all fields present. Writes a task_input.json file to the ai-orchestrator folder with the payload. Calls execAsync('npx ts-node --transpile-only src/agent.ts --execute-task', { cwd: orchestratorPath, timeout: 120_000 }). Parses stdout for TASK_RESULT:{...} JSON. Returns { success, logs[], taskId, txHash, daRootHash, result }.

CREATE api/get-tasks/route.ts

GET handler. Reads query param: agentId. Uses ethers to query the TaskExecuted events from AgentRegistry on OG Galileo (filter by agentId). Returns the last 10 tasks as an array of { taskId, taskType, result, txHash, daRootHash, executedAt }. Use the same ethers provider setup as get-agents/route.ts.

UPDATE api/spawn-agent/route.ts

Add ownerAddress to the request body: `const { ownerAddress } = await req.json()`.
Pass it to the orchestrator via environment variable:
`env: { ...process.env, SPAWN_OWNER_ADDRESS: ownerAddress }`
Also parse `SPAWN_RESULT:{...}` from stdout and return it in the response, including the agentId and txHash.

MODULE 4 OF 5

Wallet Connect — RainbowKit + wagmi integration

TYPE: Development

FILES: layout.tsx + WalletProvider.tsx +
header update

Install: `npm install @rainbow-me/rainbowkit wagmi viem @tanstack/react-query`

STEP 1 — Install dependencies

```
cd mission-control npm install @rainbow-me/rainbowkit wagmi viem @tanstack/react-query
```

STEP 2 — Create mission-control/src/components/WalletProvider.tsx

This is a client component ('use client' at top). It wraps the app with: - WagmiProvider with a config for OG Galileo chain (chainId: 16602) - RainbowKitProvider with darkTheme() - QueryClientProvider from @tanstack/react-query The OG Galileo chain config object: `const galileo = { id: 16602, name: 'OG Galileo Testnet', network: '0g-galileo', nativeCurrency: { name: '0G', symbol: '0G', decimals: 18 }, rpcUrls: { default: { http: ['https://evmrpc-testnet.0g.ai/'] } }, blockExplorers: { default: { name: 'Chainscan', url: 'https://chainscan-galileo.0g.ai' } }, testnet: true, }` Create a wagmi config: `const config = createConfig({ chains: [galileo], connectors: [injected()], transports: { [galileo.id]: http() } })`

STEP 3 — Wrap layout.tsx

In mission-control/src/app/layout.tsx, import WalletProvider and wrap the children: `<WalletProvider>{children}</WalletProvider>` Import RainbowKit CSS at the top of layout.tsx: `import '@rainbow-me/rainbowkit/styles.css';`

STEP 4 — Add ConnectButton to the dashboard header

STYLE NOTE: Override RainbowKit default styles to match the SAK dark theme. In globals.css add: `[data-rk] { --rk-colors-accentColor: #00FFD1 !important; }` This turns the connect button cyan to match the brand palette.

fetchTaskHistory() function:

```
GET /api/get-tasks?agentId={agentId} On success: setTaskHistory(data.tasks) Call this on mount (useEffect) and after every successful execution.
```

STYLING: Match the existing SAK design system exactly. Use Space Mono for all mono text. Cyan (#00FFD1) for borders and accents. Dark card background (#0C1220). Execute button: bordered cyan, fills on hover. Task history rows: alternating #070C14 / #0C1220 backgrounds. TX hash links: purple (#9B6DFF), hover cyan, open chainscan-galileo.0g.ai/tx/{hash}.

STEP 2 — Add TaskPanel to AgentDetailModal.tsx

At the bottom of the modal content (after the Network Settlement section), add a new section: `<div className='border-t border-white/10 pt-6 mt-6'> <TaskPanel agentId={agent.id} ownerAddress={agent.owner} maxAmount={1000} /> </div>` Add ownerAddress to the agent prop type if not already present.

VERIFICATION CHECKLIST

Run these checks after all 5 modules are complete

Area	Verification step
Contract	■ npx hardhat compile passes with zero errors
Contract	■ AgentRegistry redeploys — new addresses saved in addresses.json
Contract	■ AgentRegistry ABI updated in all orchestrator and frontend files
Orchestrator	■ npx tsc --noEmit passes in ai-orchestrator/
Orchestrator	■ task-parser.ts correctly extracts amount from 'send 50 tokens'
Orchestrator	■ Running --execute-task with a test payload produces TASK_RESULT: output
API	■ /api/execute-task returns { success: true, txHash, daRootHash, taskId }
API	■ /api/get-tasks returns task history array for a real agentId
Wallet	■ ConnectButton renders in header — connects to MetaMask on 0G Galileo
Wallet	■ Spawn button shows error if wallet not connected
Wallet	■ Spawned agent shows correct owner address from connected wallet
Task UI	■ TaskPanel renders inside AgentDetailModal
Task UI	■ Typing an instruction and clicking Execute calls the API
Task UI	■ Log lines stream into the terminal during execution
Task UI	■ Completed task appears in task history with explorer link

E2E

■ Full flow: Connect wallet → Spawn agent → Open modal → Give instruction → TX confirmed on Galileo

CRITICAL NOTES FOR YOUR AI AGENT

DEPENDENCY ORDER IS STRICT

Module 1 (contract) MUST compile and deploy before Module 2 (orchestrator) because the new ABI (executeTask signature) is needed. Module 3 (API routes) depends on Module 2's CLI flag. Modules 4 and 5 can run in parallel after Module 3.

DO NOT TOUCH THESE FILES

- zk-engine/circuits/constitution.circom – circuit is already correct •
- zk-engine/circuits/circuit_final.zkey – do not regenerate •
- ai-orchestrator/src/0g-service.ts – 0G integration is working •
- ai-orchestrator/src/prover.ts – ZK proof generation is correct •
- contracts/contracts/Groth16Verifier.sol – do not touch •
- contracts/contracts/interfaces/IZKVerifier.sol – interface is correct

AFTER REDEPLOYMENT

- Update these with the new AgentRegistry address: • contracts/addresses.json •
- ai-orchestrator/.env (REGISTRY_ADDRESS=) • mission-control/.env.local
- (NEXT_PUBLIC_REGISTRY_ADDRESS=) • README.md contract addresses table

0G GALILEO CHAIN DETAILS

Chain ID: 16602 RPC: <https://evmrpc-testnet.0g.ai/> Explorer: <https://chainscan-galileo.0g.ai> EVM Version: paris (set in hardhat.config.ts – DO NOT change) Faucet: <https://faucet.0g.ai>

This prompt covers everything needed to make SAK agents genuinely useful. After completion, users connect their MetaMask wallet, spawn an agent they own on-chain, open the Agent Detail modal, type any instruction, and watch it flow through ZK proof enforcement → 0G DA memory → on-chain settlement in real time.